AI ENGINEERING COURSE

Module 17

Frontier Ideas in AI

3 lessons · Personal offline study copy

Personal offline study copy. Content © Outcome School / Amit Shekhar. Not for redistribution. See study/NOTICE.md.

Contents

Module 17: Frontier Ideas in AI

- 17.1 What is Joint Embedding Predictive Architecture (JEPA) and How Does It Work?
- 17.2 How do World Models work?
- 17.3 What is Recursive Self-Improvement (RSI)?

MODULE 17

Frontier Ideas in AI

3 lessons · ~1 min overview

In this module, we will learn the ideas that are shaping the future of AI, from models that learn an internal picture of the world to systems that improve themselves.

By the end of this module, we will understand where AI research is heading next.

LESSON 17.1

What is Joint Embedding Predictive Architecture (JEPA) and How Does It Work?

Lesson 17.1 · ~26 min · Published June 7, 2026 · https://outcomeschool.com/blog/joint-embedding-predictive-architecture-jepa



Joint Embedding Predictive Architecture (JEPA) is an idea from Yann LeCun where a model learns by predicting the hidden part of the data from the visible part. It makes this prediction in the embedding space, which means on short summaries, not on raw pixels.

In this blog, we will learn about Joint Embedding Predictive Architecture (JEPA).

This is one of the most exciting ideas in modern AI, and it comes from Yann LeCun, one of the most respected researchers in the field. Do not worry, we will learn about each part of it slowly, in very simple words. By the end, a complete beginner will understand every single word.

We will cover the following:

- How humans and animals learn by observing the world
- Yann LeCun's vision of autonomous machine intelligence
- A simple everyday analogy to build the intuition
- What does JEPA mean?
- What is an embedding or representation space?
- The problem with predicting raw pixels
- The problem with contrastive methods
- The core idea of JEPA

- The building blocks of JEPA
- The energy-based view in simple words
- How I-JEPA works (for images)
- V-JEPA and the world-model vision
- When and why JEPA matters

Let's get started.

How humans and animals learn by observing the world

Before jumping into JEPA, we must know how we, as humans, learn in the first place.

Let's say a small baby is sitting at a table. The baby slowly pushes a glass towards the edge. At first, the glass stays on the table. But when the baby pushes it a little more, the glass falls down.

The baby just learned something deep. The baby learned that objects fall when they go off an edge.

Nobody taught this with words. Nobody handed the baby a textbook. The baby simply observed the world and figured out how the world works.

This is how humans and animals learn most things. We watch. We observe. And we build an understanding of the world inside our head.

A cat knows that if it jumps, it will land somewhere. A child knows that if a ball rolls behind a sofa, the ball still exists. We all carry a kind of inner sense of how the world behaves.

This inner sense lets us imagine what will happen next, before it even happens. And this simple human ability is the seed from which the whole idea of JEPA grows.

Now, the question is: can a machine also learn like this? Can a machine learn how the world works just by observing, the way a baby does?

This is exactly the dream that researchers are chasing.

Yann LeCun's vision of autonomous machine intelligence

Yann LeCun is one of the most respected researchers in Artificial Intelligence. He served as Chief AI Scientist at Meta, where this JEPA research was done, and he is now building world models at his own venture, Advanced Machine Intelligence (AMI) Labs.

In June 2022, he wrote a paper called "A Path Towards Autonomous Machine Intelligence."

In this paper, he shared a big vision. He wants to build machines that learn a model of the world by observation, just like a baby does. Such a machine could then imagine the result of its actions and plan towards a goal.

He calls this **autonomous machine intelligence**, which means a machine that learns on its own from observation and acts with common sense.

For this dream, he needed a core building block, a way for a machine to learn by predicting. And the building block he proposed is JEPA.

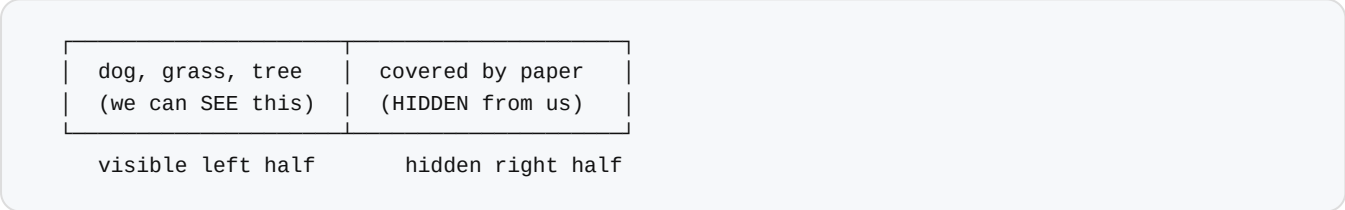
So, here comes JEPA into the picture. Let's slowly understand it, starting with a simple everyday picture in our head.

A simple everyday analogy

Let's say we are looking at a photo of a street.

We can see the left half of the photo clearly. There is a dog, some grass, and a tree. The right half of the photo is covered with a sheet of paper. We cannot see it.

Let's picture it as below:



Here, we can see the photo split into two halves. The left half is visible, so we can see the dog, the grass, and the tree. The right half is hidden behind a sheet of paper. We can still guess the gist of what is hidden, but not the exact pixels.

Now, the question is: can we guess what is behind that paper?

Yes, we can. Our brain says, "There is probably more grass, maybe more of the tree, and the dog's tail."

But here is the important part. We do not guess the exact pixels. We do not know the exact shade of every blade of grass. We do not know the exact position of every leaf. We do not know the tiny shadows or the exact lighting.

We only guess the gist. The idea of what is hidden. "More grass, a tail, a bit of tree."

This is exactly how JEPA thinks.

JEPA looks at one part of the data, and it predicts the gist of the hidden part, not every exact pixel. This single idea is the heart of everything we will learn today.

Now, let's understand the name.

What does JEPA mean?

JEPA stands for **Joint Embedding Predictive Architecture**.

The name sounds heavy, so let's break it into small pieces.

> JEPA = Joint + Embedding + Predictive + Architecture

Let's understand each word in plain language.

Joint means two things working together. Here, we have two inputs. One is the part we can see (the context). The other is the part we want to guess (the target). Both are handled together, side by side.

Embedding means a short, meaningful summary of something. We will explain this properly in the next section, because it is the most important word here.

Predictive means guessing. The model predicts the summary of the hidden part from the summary of the visible part.

Architecture simply means the design, or the structure of how the pieces are connected. It is the blueprint of the system.

So, in simple words, JEPA is a design where the model takes a summary of the visible part and predicts the summary of the hidden part.

Now, the word "summary" is doing a lot of work here. So, before going further, we must know what an embedding really is.

What is an embedding or representation space?

This is the most important idea in the whole blog. So, let's go slow.

An embedding is a short list of numbers that captures the meaning of something.

In simple words, an embedding is a compact summary.

Let's say we have a photo of a dog. The raw photo is millions of tiny colored dots called **pixels**. A pixel is just one tiny dot of color in an image. A normal photo has millions of them.

These millions of pixels are very detailed and very messy. Most of that detail does not matter. The exact noise in the background, the exact lighting, the exact texture of the fur, these tiny things do not really tell us "this is a dog."

Now, instead of millions of pixels, suppose we describe the photo with a short list of numbers that captures the meaning: "this is a dog, it is brown, it is on grass, it is facing left."

That short, meaningful list of numbers is called an **embedding**.

The space where all these summaries live is called the **representation space** or **embedding space**. Means, it is the world of summaries, instead of the world of raw pixels.

Here is the key benefit. In the embedding space, similar things sit close together. The embedding of a dog photo and the embedding of another dog photo will be close to each other. The embedding of a dog and the embedding of a car will be far apart.

So, an embedding throws away the messy, unimportant details and keeps the meaning.

Here, the raw pixels are the messy full detail, and an embedding is the clean meaningful summary.

Now that we understand embeddings, let's see the problem that JEPA is trying to solve.

The problem with predicting raw pixels

For a long time, many AI models learned by predicting the raw input.

Let's understand this with an example. We cover half of an image, and we ask the model to redraw the missing half, pixel by pixel. The model that does this is called a **generative model**, because it generates, or creates, the actual content.

We have a detailed blog on diffusion models, a popular family of generative models, that explains how they create images in depth.

This sounds nice, but here is the catch.

To redraw the missing half perfectly, the model must guess every tiny detail. The exact texture of the grass. The exact noise in the sky. The exact lighting on every leaf.

But these tiny details are unpredictable. There is no way to know the exact random noise. So the model wastes a huge amount of effort trying to guess details that nobody can guess.

For the sake of understanding, predicting raw pixels is like being asked to redraw a photograph from memory, perfectly, including every random speck of dust. That is a hopeless task.

So, this approach spends most of its energy on the wrong thing. It focuses on the unpredictable detail instead of the predictable meaning.

The issue with this approach is wasted effort on details that do not matter. Let's see how the next approach tries to solve this issue.

The problem with contrastive methods

So, researchers thought of another approach, called **contrastive learning**.

In simple words, contrastive learning shows the model two versions of the same image and teaches it, "these two are the same thing, pull them close." Then it shows two different images and teaches it,

"these two are different, push them apart."

The "different" examples are called **negative examples**. They are the things the model must push away.

This works, but here is the catch.

To work well, this method usually needs a lot of these negative examples. We have to keep collecting and comparing many "this is not a match" pairs. That is extra work, and it can be hard to manage.

It also often needs **data augmentation**, which means we manually create new versions of an image by cropping it, flipping it, or changing its colors. Designing these tricks by hand is fiddly, and they may not fit every kind of data.

So, contrastive learning learns by comparing, but it carries the burden of many negative examples and hand-made tricks.

The issue with this approach is the extra burden of negative examples and hand-made tricks. We needed a solution for that, and JEPA was introduced to solve this problem.

The core idea of JEPA

Let's put the two problems side by side. Generative models predict raw pixels, which wastes effort on unpredictable detail. Contrastive models compare images, which needs many negative examples and hand-made tricks.

JEPA takes a smarter path.

JEPA predicts in the embedding space, not in the pixel space.

In simple words, JEPA does not redraw the hidden part. It only predicts the summary of the hidden part.

Let's go back to our street photo. The left half is visible. The right half is hidden.

A generative model would try to redraw the right half, pixel by pixel. That is hard and wasteful.

JEPA does something different. It looks at the visible left half, makes a summary of it, and then predicts the summary of the hidden right half. It never redraws the pixels. It only guesses the meaning.

Let's picture this difference as below:

```
Generative model:
visible part → model → redraw every hidden PIXEL   (hard, wasteful)
JEPA:
visible part → model → predict the SUMMARY only    (clean, efficient)
```

Here, we can see the key difference in one picture. A generative model must redraw every hidden pixel, which is hard and wasteful. JEPA only predicts the summary of the hidden part, which is clean and efficient. Same input, but a much simpler goal.

This is powerful for one big reason. When we predict only the summary, we are allowed to ignore the messy, unpredictable details. We do not have to guess the exact random noise. We only have to guess the meaning, which is the part that can actually be guessed.

This is why JEPA is called **non-generative**. It does not generate, or create, the hidden content. It only predicts the abstract summary of it.

So, JEPA keeps the good part of both older methods and drops their pain.

From the generative method, JEPA keeps the good part. It learns the relationship between the visible part and the hidden part, by predicting the hidden part from the visible part. But it drops the pain of that method, which was redrawing every raw pixel and wasting effort on unpredictable detail.

From the contrastive method, JEPA keeps the good part. It works in the clean world of summaries, instead of the messy world of raw pixels. But it drops the pain of that method, which was the need for many negative examples and hand-made tricks like cropping and flipping.

So, in simple words, JEPA predicts the hidden part from the visible part, just like a generative model, but it does this prediction in the clean world of summaries, just like contrastive learning, without carrying the burden of either one.

Now, let's understand the building blocks that make this happen.

The building blocks of JEPA

JEPA has a few simple parts. Let's meet each one.

We will call the visible part **x** (the context), and the hidden part **y** (the target).

The x-encoder. An **encoder** is a machine that takes a raw input and turns it into an embedding, which is the summary we learned about earlier. The x-encoder takes the visible context x and produces its summary. We call this summary `s_x`.

The y-encoder. Similarly, the y-encoder takes the hidden target y and produces its summary. We call this summary `s_y`.

The predictor. This is the guessing machine. It takes the summary of the visible part, `s_x`, and predicts the summary of the hidden part. We call this prediction `s_y-hat`. The little "hat" just means "our guess of it."

But here, a small question arises. There can be many hidden parts, sitting in different places. So how does the predictor know which hidden part to guess right now? This is why we also give the predictor the **position** of the hidden part.

The position simply means the location of the hidden part, like the top-right corner or the bottom-left block. It tells the predictor, "guess the summary for this exact spot." We feed the same visible summary `s_x`, but we change the position to ask about each hidden part one by one.

A simple way to picture it. Imagine we cover a few windows of a house and ask a friend to guess what is behind each one. The friend's view of the rest of the house stays the same, but we must point and say "guess this window" or "guess that window." That pointing is the position.

So the flow is simple. The x-encoder summarizes what we see. The predictor, told which position to look at, guesses the summary of what is hidden there. Then we compare our guess `s_y-hat` with the real summary `s_y` from the y-encoder. If they are close, we did well.

Now, there is one more clever piece.

The latent variable z. Sometimes, the hidden part has information that the visible part simply cannot tell us. For example, from the left half of a photo, we cannot know whether it is raining in the right half. That information is just not there in the context.

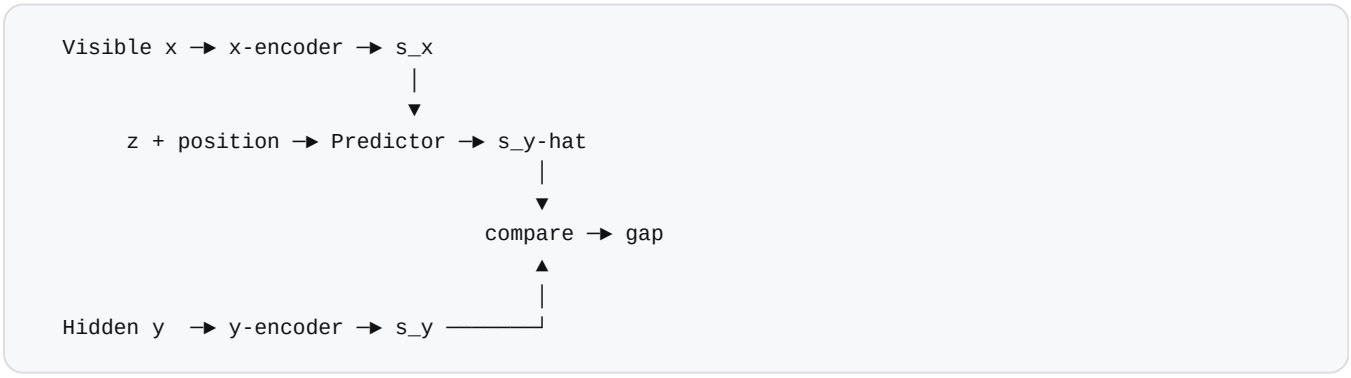
So how can the predictor guess something it has no clue about? The answer is the latent variable z.

The latent variable z is like a small extra note that carries the missing clue. It tells the predictor, "by the way, in the hidden part, it is raining." With this small note, the predictor can handle situations where the hidden part is not fully decided by the visible part.

In simple words, z handles uncertainty. It lets the model say, "the hidden part could be this, or it could be that," instead of being forced into one fixed answer.

Now, we have understood the parts: the x-encoder, the y-encoder, the predictor, the position of the hidden part, and the latent variable z.

Let's put all of these parts together into one simple picture as below:



Here, we can see the two tracks of JEPA in one picture. On the top track, the visible part `x` goes into the x-encoder and becomes its summary `s_x`. This summary, together with the latent variable `z` and the position of the hidden part, goes into the predictor, which produces the guessed summary `s_y-hat`. On the bottom track, the hidden part `y` goes into the y-encoder and becomes the real summary `s_y`. Finally, we compare our guess `s_y-hat` with the real summary `s_y`. The gap between them is the error, and the whole job of training is to make this gap small. In the next section, we will see that this gap even has a name.

Note: This two-track picture is the simple, conceptual view of JEPA. It shows the hidden part `y` going into the y-encoder, just to keep the core idea clear. In a real model like I-JEPA, which we will see soon, the y-encoder actually looks at the whole image first and then picks out the summary of the hidden part from its output. The end result is the same, a summary `s_y` of the hidden part, so the picture stays true to the idea.

Let's now look at one more way researchers describe JEPA, because it is simple and useful.

The energy-based view in simple words

JEPA is also described as an **energy-based model**.

Think of energy as a **wrongness score**.

A low score means "these two fit well together." A high score means "these two do not fit together."

In JEPA, the energy is simply the gap between our guessed summary `s_y-hat` and the real summary `s_y`. Both of these summaries come from parts of the same input, for example two regions of one image, not from matching an image to some text.

A small gap means our guess was good, which means low energy. A big gap means our guess was bad, which means high energy.

Note: Please remember, low energy means a good, matching pair. High energy means a bad, mismatched pair.

Just for the sake of a general intuition about energy-based models, low energy is given to things that belong together, and high energy is given to things that do not. In JEPA specifically, "belong together" means the predicted summary `s_y-hat` is close to the real summary `s_y`. Training the model means making this gap small for real, matching pairs.

So, in plain words, training JEPA means teaching it to give a low wrongness score to pairs that truly belong together.

This is how JEPA learns. Now, let's see a real model that actually puts all of this into practice.

How I-JEPA works (for images)

The first real version of JEPA for images is called **I-JEPA**, where the "I" stands for Image. It was released by Meta AI in 2023.

Before that, we must know one simple idea behind all of this, called **self-supervised learning**. In simple words, self-supervised learning means the model teaches itself by hiding a part of the data and then trying to predict that hidden part. Nobody has to sit and label the data by hand. The data itself

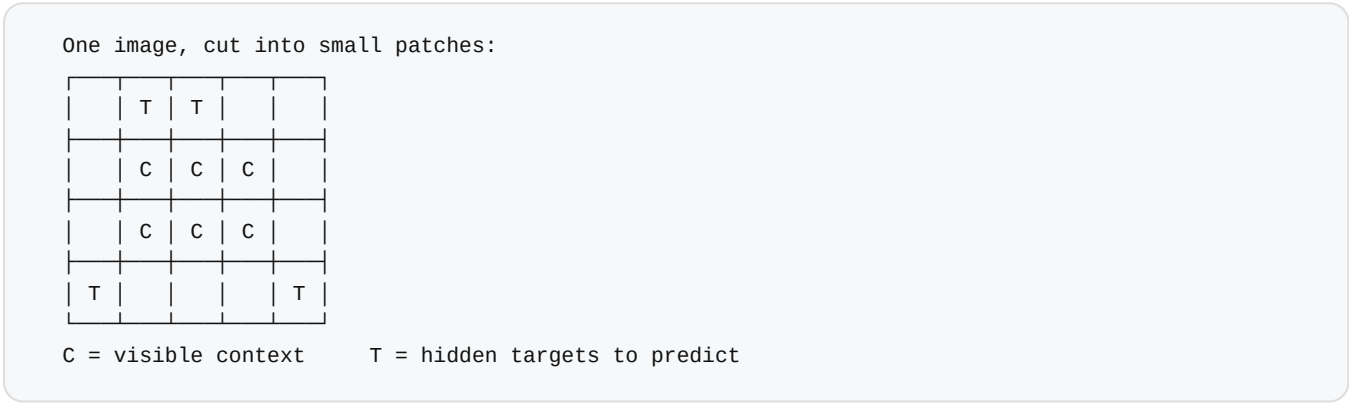
becomes the teacher. This is exactly what lets these models learn from huge amounts of raw images and videos.

Now, let's understand I-JEPA step by step, using everything we have learned.

First, I-JEPA takes one image. It cuts the image into small square pieces called **patches**, and treats them like a sequence. The model that processes these patches is called a **Vision Transformer**, often shortened to ViT. We do not need the deep details. We can simply think of it as the engine that reads the image patches.

Then, I-JEPA picks the parts to work with. It chooses one larger visible block, called the **context block**. This is our x, the part we can see. It also chooses a few hidden blocks, called **target blocks**. These are our y, the parts we must predict. To make the task fair and not too easy, any patch that appears in a target block is removed from the context block.

Let's picture this as below:



Here, we can see one image cut into small patches. The patches marked `C` form the context block, which is the visible part we feed to the context encoder. The patches marked `T` are the target blocks, which are the hidden parts the model must predict. Notice that the target patches are kept out of the context, so the model cannot simply copy the answer. This picture is simplified just for the sake of understanding, but the idea is exactly this.

After that, three pieces go to work, and we will recognize them.

The **context encoder** takes the visible context block and produces its summary. This is our x-encoder.

The **target encoder** runs over the whole image and produces a summary for every patch. The summaries for the chosen target blocks are then picked out from this output. This is our y-encoder. One important detail here is that the target blocks are taken from the encoder's output, not by hiding patches at the input. The encoder always sees the full image first, and only afterward do we select the parts we care about.

The **predictor** takes the context summary, along with the position of each hidden block, and predicts the summary of each target block. It never draws any pixels. It only predicts summaries, exactly as we discussed.

Finally, the model compares its predicted summaries with the real summaries from the target encoder. If they are close, the model is doing well. The model learns by making them closer over time.

Now, there is one very clever trick we must understand. It is called avoiding **collapse**.

Here is the danger. If both encoders are free to learn anything, they can cheat. They can decide to output the same boring summary for everything. Then the prediction always matches, the wrongness score is always low, and the model has learned nothing useful. This lazy cheating is called **representation collapse**.

So, how does I-JEPA stop this cheating? The answer is a smart, uneven design.

The context encoder learns normally and updates quickly at every step. But the target encoder does not learn directly. Instead, its values slowly follow the context encoder, like a calm, slow copy of it. This slow following is called an **exponential moving average**, often shortened to EMA. Means, the target encoder updates a little bit at a time, very gently.

On top of this, I-JEPA uses a **stop-gradient** on the target side. In simple words, this means the learning signal is not allowed to flow into the target encoder. The target encoder acts like a fixed answer key during the lesson. The student cannot secretly change the answer key to make its own answers look correct.

Let's picture this uneven design as below:

```
context encoder  ──►  learns fast, updates every step
      │
      │  slow copy (EMA), no learning flows back (stop-gradient)
      ▼
target encoder   ──►  follows gently, acts as a fixed answer key
```

Here, we can see the uneven design that stops the collapse. The context encoder learns fast and updates at every step. The target encoder does not learn on its own. It only follows the context encoder slowly through the EMA, and the stop-gradient makes sure no learning signal flows back into it. Because the target encoder stays a step behind and acts as a fixed answer key, the two encoders cannot agree to cheat together, so the collapse is avoided.

This uneven design, a fast context encoder plus a slow target encoder with a stop-gradient, is what stops the collapse. And here is the beautiful part. It does all this without any negative examples and without any hand-made data augmentation. It avoids the burden that contrastive methods carry.

There is one more reason to love I-JEPA. Because it predicts summaries and not pixels, it is very efficient. A large I-JEPA model was trained on the well-known ImageNet dataset, which is a huge collection of images used to train and test vision models. During this training, I-JEPA used only the raw images and not their labels, because it teaches itself in the self-supervised way we learned about. It was trained using sixteen powerful computer chips called GPUs, in under seventy-two hours. That is far less effort than many older methods. The summaries it learns are strong and useful, and they transfer well to many other tasks like recognizing objects, counting objects, and judging depth.

This is how I-JEPA brings the JEPA idea to life for images.

This was all about images. Now, let's move to video and the bigger vision.

V-JEPA and the world-model vision

After images, the same idea was extended to video. This version is called **V-JEPA**, where the "V" stands for Video. It was released by Meta AI in February 2024.

The idea is the same. V-JEPA hides some regions of a video and predicts their summaries from the visible regions, all in the embedding space, never in pixels. It learned from a very large collection of unlabeled videos. Unlabeled simply means nobody had to sit and tag what is in each video, so the model learns from raw video at a huge scale.

Then came **V-JEPA 2**, released by Meta in June 2025. This one is bigger, and it points toward a deeper goal called a **world model**.

So, what is a world model? Let's understand with a simple example.

Suppose we are pouring water into a glass. Before the glass overflows, we already know in our head that it is about to overflow, so we stop. We did not have to actually spill the water to know it. We have an internal sense of how the world works.

A world model is exactly that, but inside a machine. It is an internal sense of how the world behaves, so the machine can imagine the result of an action before doing it.

V-JEPA 2 takes a step toward this. A special version of it, trained on a small amount of robot video, was able to control a real robot arm to pick up objects and place them, in labs where it had never trained before, just by being given a goal image. Means, we show the robot a picture of the finished result, and it plans the steps to get there by imagining the outcomes in the embedding space.

This connects back to the bigger vision we started with. This is exactly what Yann LeCun described in his 2022 paper, "A Path Towards Autonomous Machine Intelligence," a step toward machines that have common sense. The idea is that a machine can learn how the world works just by watching, predict what will happen next, and then plan toward a goal, just like the baby with the glass.

Note: JEPA is not a magic replacement for everything. On hard physical and reasoning tests, these models still do not match humans. JEPA is meant to be complementary, adding the kind of intuitive understanding of the world that text-only models often lack.

There is a whole family growing from this idea. Beyond images and video, there are versions for other kinds of data too, like audio, motion, and more. They all share the same heart: predict the summary of the hidden part from the summary of the visible part.

When and why JEPA matters

Now, let's bring it all together and see why this matters.

JEPA matters because it focuses on meaning, not on messy detail. By predicting summaries in the embedding space, it skips the impossible task of guessing every unpredictable pixel, and it spends its effort on the part that can actually be learned.

It matters because it is efficient. Predicting summaries is lighter work than redrawing pixels, so it trains faster and cheaper.

It matters because it is clean. It avoids the heavy burden of negative examples and hand-made augmentations that older methods needed.

It also matters because it learns a different kind of knowledge. Large language models, like the ones that power chat assistants, read a huge amount of text. But text alone often misses simple physical sense, like the fact that a glass falls when it goes off a table. JEPA learns this kind of intuitive understanding by watching the world. So, JEPA is complementary to large language models, not a replacement for them.

And it matters because it points toward a bigger dream. By learning how the world works from simple observation, JEPA is a step toward machines that can imagine, plan, and act with a bit of common sense.

So, let's recall the one idea we started with. We look at part of a scene, and in our head we predict the gist of the hidden part, not every exact pixel. That simple human intuition is the soul of JEPA, and that is what makes it such a beautiful idea in modern AI.

Frequently Asked Questions

Is JEPA a generative model?

No. JEPA is non-generative. It does not redraw or create the hidden content pixel by pixel. It only predicts the summary of the hidden part in the embedding space. This lets it ignore the messy, unpredictable details and focus on the meaning, which is the part that can actually be guessed.

Does JEPA need labeled data to learn?

No. JEPA learns in a self-supervised way. It hides a part of the data and tries to predict the summary of that hidden part, so the data itself becomes the teacher. For example, I-JEPA was trained on ImageNet using only the raw images, not their labels.

Does JEPA need negative examples like contrastive learning?

No. JEPA avoids collapse without any negative examples and without hand-made data augmentation. In I-JEPA, a fast context encoder and a slow target encoder that follows it through an exponential moving average, along with a stop-gradient, stop the model from cheating.

Will JEPA replace large language models?

No. JEPA is complementary to large language models, not a replacement for them. LLMs learn from a huge amount of text, but text alone often misses simple physical sense. JEPA adds the kind of intuitive understanding of the world that comes from watching it.

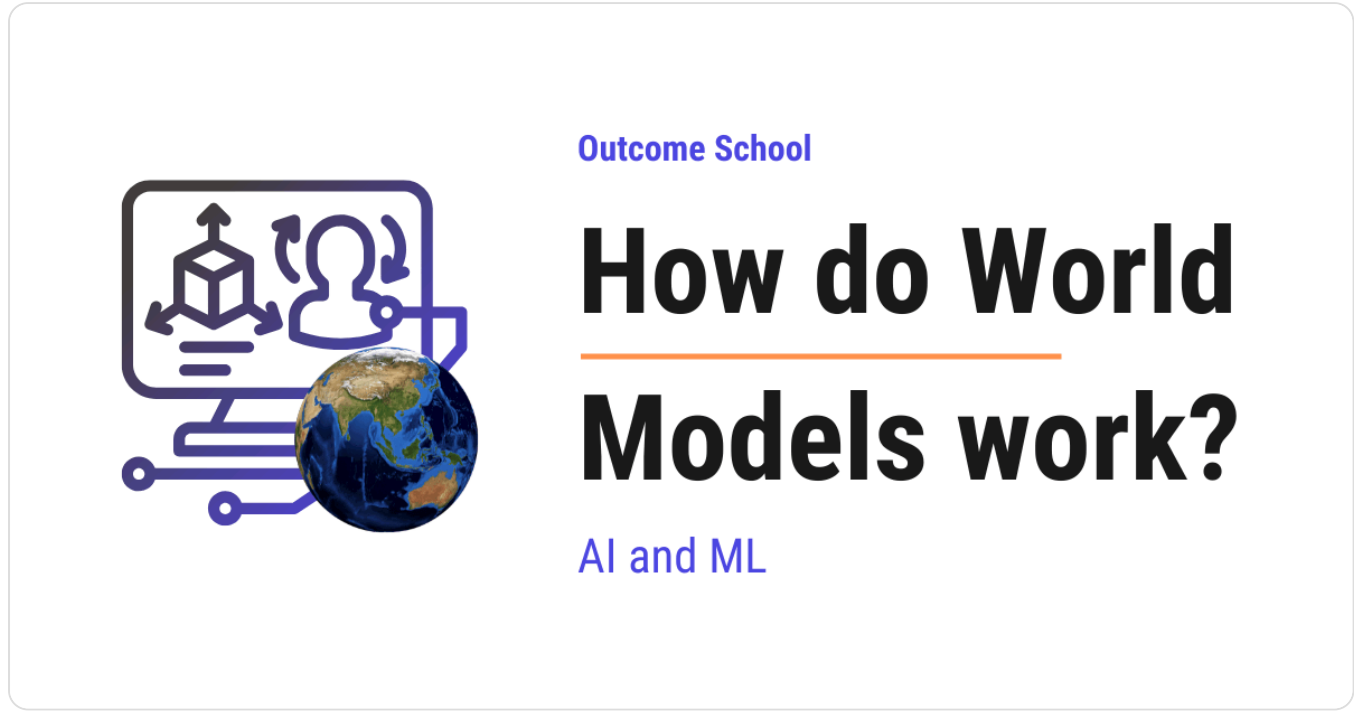
Do JEPA models understand the physical world as well as humans?

No. On hard physical and reasoning tests, these models still do not match humans. JEPA is not a magic replacement for everything. It is a step toward machines that can learn how the world works by observation, then imagine and plan with a bit of common sense.

LESSON 17.2

How do World Models work?

Lesson 17.2 · ~21 min · Published June 20, 2026 · https://outcomeschool.com/blog/how-do-world-models-work



A World Model is an AI that learns an approximate internal copy of how an environment behaves, so that it can predict what happens next, given the current state and an action.

In this blog, we will learn about how World Models work. We will also see why we need them, how they actually learn an internal picture of an environment, and where they are used in real systems like robotics, game-playing, and video generation.

We will cover the following:

- What is an environment, a state, and an action?
- What is a World Model?
- The human analogy: imagining a move before making it
- Why we need a World Model
- How a World Model learns: predicting the next state
- The latent state: compressing what we see
- Imagining the future: rolling out without touching the real world
- Dreamer-style agents that plan inside the model
- World Models and predicting the future
- World Models in the real world

Let's get started.

What is an environment, a state, and an action?

Before we talk about World Models, we must first understand three simple words: environment, state, and action. These three words come up again and again, so let's make them very clear.

An environment is the world that our AI lives in and interacts with.

In simple words, the environment is the surroundings. For a robot, the environment is the room it walks around in. For a game-playing AI, the environment is the game itself. For a self-driving car, the environment is the road, the other cars, and the traffic lights.

A state is a snapshot of the environment at one moment in time.

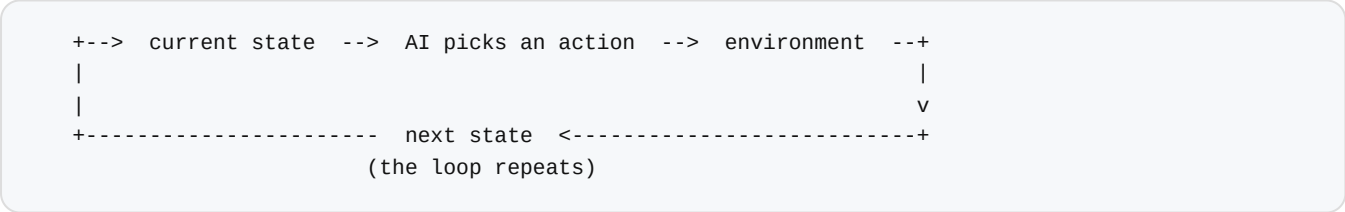
In simple words, the state is "how things are right now". Let's say we are playing a video game. The state is the current screen: where our character is standing, where the enemies are, and how much health we have left. If anything moves, we get a new state.

An action is a choice the AI makes that changes the state.

In simple words, an action is a move. In a game, an action could be "jump", "move left", or "shoot". For a robot, an action could be "turn the wheel" or "lift the arm".

So, here is the simple loop that ties them together. The AI looks at the current state, picks an action, and the environment gives back a new state. Then it looks at the new state, picks another action, and so on.

We can picture this loop as below:



Here, we can see that the AI and the environment keep taking turns. The AI acts, the environment responds with a new state, and this continues step after step. This loop is the foundation we needed. Now we are ready to understand the problem.

What is a World Model?

Now that we know what a state and an action are, let's understand a World Model.

A World Model is an AI that learns an approximate internal copy of how an environment behaves, so that it can predict what happens next, given the current state and an action.

Let's decompose the term to make it stick.

World Model = World + Model.

The "world" is the environment we just talked about. A "model" is simply a small internal copy or imitation of something real. So a World Model is an approximate internal copy of the world that lives inside the AI. It is not a perfect copy, but it is good enough to be useful.

In simple words, a World Model is a little simulator that the AI builds inside its own head. We can ask it a question like "if I am in this state and I take this action, what happens next?" and it gives back an answer: the next state, and how good or bad that outcome is.

Before we go further, let's quickly understand one more word: **reward**. A reward is a number that tells the AI how good an outcome is. A high reward means "that was good, do more of that". A low or negative reward means "that was bad, avoid it". For a game, scoring a point gives a reward. Falling into a pit gives a negative reward.

So, a World Model takes two things as input, the current state and an action. From these, it predicts what comes next. The main thing it predicts is the next state. Many World Models, especially the ones used in Reinforcement Learning, also predict the reward. In this blog, we will focus on these, so from here on we will talk about predicting the next state and the reward together.

This is how a World Model works at a high level. Now, let's make it feel real with an everyday example.

The human analogy: imagining a move before making it

The best way to learn this is by taking an example. And the best example is something we all do every single day, without even thinking about it.

Let's say we are playing chess. Before we actually touch a piece, what do we do? We imagine. We think, "If I move my knight here, then my opponent will probably move their bishop there, and then I will be in trouble." We play the move in our head first. We look at the imagined outcome. And only if it looks good, we make the real move.

We did not touch the board to find this out. We ran a little simulation inside our mind.

That little simulation inside our mind is exactly a World Model.

Let's take one more example. Suppose we are about to pour water into a glass. Before we tilt the bottle, we already have a rough idea of what will happen: the water will flow out, the level in the glass will rise, and if we tilt too much, it will overflow. We know all of this without spilling a single drop, because we carry an internal model of how water behaves.

So, a World Model gives an AI this same ability: the power to imagine the outcome of an action before taking it in the real world.

This is the heart of the whole idea. Now, the question is, why do we even need this? Let's see.

Why we need a World Model

To understand why we need a World Model, let's first look at how an AI learns without one.

The usual way for an AI to learn a task is by trial and error. It tries an action, sees what happens, and slowly figures out which actions are good and which are bad. This way of learning by trial and error and rewards is called **Reinforcement Learning**.

In simple words, Reinforcement Learning is learning from experience: try something, get a reward or a penalty, and remember what worked.

But here is the catch. To learn this way, the AI has to actually try things in the real environment, again and again, thousands or even millions of times. And that brings real problems.

This causes three real problems:

- **It is slow.** Trying millions of actions in the real world takes a very long time.
- **It is expensive.** If our AI is a real robot, every trial means real motors moving, real battery used, and real wear and tear.
- **It is risky.** A real robot that learns by random trial and error can fall, break things, or hurt itself while figuring things out.

So, learning directly in the real world is wasteful and sometimes dangerous.

We can compare the two ways of learning as below:

```
WITHOUT a World Model            WITH a World Model
---------------------            ------------------
AI                               AI
 |                                |
 | try in REAL world              | try inside the MODEL
 v                                v
real environment                 internal simulator
 |                                |
 | slow, expensive, risky         | fast, cheap, safe
 v                                v
new state + reward               predicted state + reward
 |                                |
 | repeat millions of times       | repeat millions of times
 | (in the slow real world)       | (all in imagination)
 v                                v
learned behavior                 learned behavior
```

Here, we can notice that both paths reach the same goal of a learned behavior, but the path on the left keeps paying the slow, costly, risky price of the real world on every single try, while the path on the right does all that practice inside a cheap, safe internal simulator.

Now, think back to the chess example. We did not move real pieces a million times to learn. We imagined moves in our head and learned from those imagined outcomes. That is the trick we want to give the AI.

So, here comes the World Model to the rescue. Once the AI has an internal model of the world, it can practice inside that model instead of practicing in the real world. We will see exactly how this saves so much effort in a moment.

How a World Model learns: predicting the next state

Now, let's understand how a World Model actually learns. This is simpler than it sounds.

The World Model learns by watching experience and learning to predict what comes next.

Let's walk through it step by step.

Step 1: First, we let the AI interact with the environment for a while and record what happens. Each recorded piece is a small story: "I was in this state, I took this action, and then this next state happened, and I got this reward." We collect many such stories.

Step 2: Then, we train the World Model on these recorded stories. We show it the state and the action, and we ask it to guess the next state and the reward. At first, its guesses are wrong.

Step 3: After that, we compare its guess with what actually happened. If the guess is wrong, we nudge the model to do better next time. We repeat this for all the recorded stories, over and over.

Finally, after enough practice, the World Model becomes good at this. Given a state and an action, it predicts the next state and reward accurately. And the best part is that it works even for situations it has not seen exactly before, because it has learned the general rules of the environment rather than just memorizing each recorded story.

We can picture what the trained World Model does as below:



Here, we can see that the World Model takes the current state and the action on the left, runs them through what it learned, and produces the predicted next state and reward on the right. It has essentially learned the rules of the environment, just by watching enough examples, and that too without anyone hand-coding those rules.

This is how a World Model learns. Now, there is one more clever piece that makes it really powerful. Let's understand it.

The latent state: compressing what we see

Here is a small problem we must deal with first.

In many environments, what the AI actually sees is raw and huge. For example, a game screen or a camera image is just a giant grid of pixels. A single image can have millions of numbers in it, one for each tiny color dot. Predicting the next full image, pixel by pixel, would be enormous and wasteful.

Most of those pixels do not even matter. In a game, the color of the sky in the background does not change what we should do. What matters is where our character is, where the enemies are, and a few other important details.

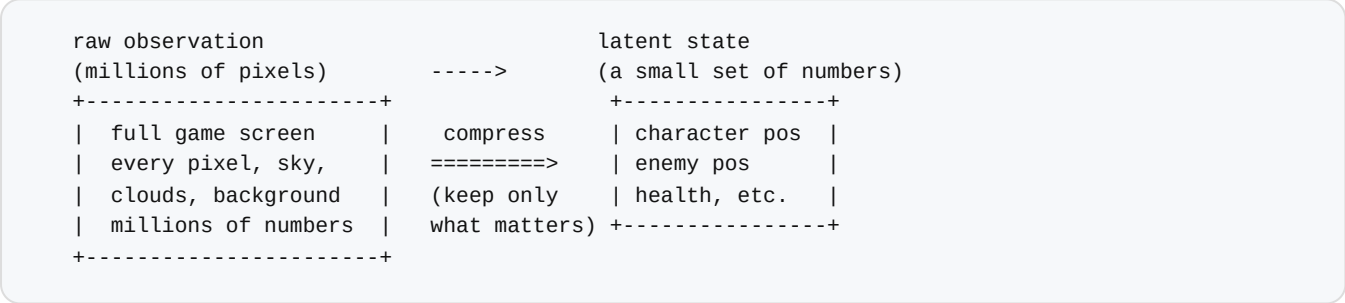
So, the World Model does something smart. It compresses the raw observation into a small, compact summary that keeps the important information and throws away the rest. One thing to note here is that the World Model decides for itself what is important. It keeps whatever helps it predict the future well, which is mostly, but not always, the same as what we humans would have picked.

This compact summary is called the **latent state**.

In simple words, the latent state is a short description of "what is really going on", squeezed down from the big raw image into a small set of numbers.

Let's understand it with an analogy. Suppose a friend watches a football match and then describes it to us in two sentences: "Our team is attacking, the striker has the ball near the goal, and the defender is closing in." Our friend did not repeat every single frame of the match. They compressed the whole scene into a tiny, meaningful summary. The latent state is exactly that kind of tiny, meaningful summary.

We can picture the compression as below:



Here, we can see that the World Model takes the huge raw screen on the left and squeezes it into a tiny latent state on the right that holds only the things that matter for predicting what comes next. Because the latent state is small, the model can work with it quickly and cheaply.

So, from now on, the World Model does its predicting in this compact latent space. It takes a latent state and an action, and it predicts the next latent state and the reward. This makes everything faster and easier to learn.

This idea of squeezing a raw observation down into a small set of meaningful numbers is exactly what an encoder learns to do in a Variational Autoencoder - we have a detailed blog on Variational Autoencoders that explains how that compression works.

This is how a World Model handles big, messy inputs like images. Now, let's get to the most exciting part: imagining the future.

Imagining the future: rolling out without touching the real world

Now we have all the pieces. Let's bring them together for the big payoff.

Once the World Model can predict the next latent state from a current latent state and an action, we can chain these predictions together.

In simple words, we can imagine a whole sequence of future steps, all inside the model, without touching the real environment even once.

Predicting the future directly in this compact latent space, rather than in raw pixels, is also the core idea behind JEPA - we have a detailed blog on Joint Embedding Predictive Architecture (JEPA) that covers this in depth.

This chaining of predicted steps is called a **rollout**. A rollout is just "imagine step one, then step two, then step three, and so on", entirely in imagination.

Let's walk through how a rollout works.

Step 1: We start with the current latent state.

Step 2: We pick an action. We feed the latent state and the action into the World Model, and it predicts the next latent state and the reward.

Step 3: Now we take that predicted next state, pick another action, and feed it into the World Model again. It predicts the state after that.

Finally, we keep repeating this for many steps. We have now imagined a full future, several moves ahead, without taking a single real action.

We can picture a rollout as below:

```
IMAGINED FUTURE (all inside the World Model, no real environment)
latent state 0
      | action A
      v
latent state 1  (+ reward)
      | action B
      v
latent state 2  (+ reward)
      | action C
      v
latent state 3  (+ reward)   ...and so on
```

Here, we can see that starting from the current latent state, the model keeps predicting the next state for each chosen action, building a chain of imagined moments. At every step it also predicts the reward, so the AI can see how good this imagined path turned out to be.

This is the same thing we did with chess. We imagined a sequence of moves and their outcomes in our head before playing for real.

And here is the huge benefit. Because all of this imagining happens inside the model, the AI can practice millions of times very quickly and cheaply, without breaking any real robots and without waiting for the slow real world. This property is called being **sample-efficient**. In simple words, sample-efficient means the AI learns a good behavior from very few real experiences, because it squeezes a lot of extra practice out of its imagination.

But, here is one catch we must keep in mind. The World Model is not perfect, so every prediction it makes carries a small error. When we chain many predictions together in a long rollout, these small errors add up step after step. So the further into the future we imagine, the less reliable our imagined future becomes. This is why, in practice, these rollouts are usually kept short, and the AI keeps going back to the real environment to check and correct its model.

So, the main problem is largely solved. The AI no longer needs to learn everything the slow, costly, risky way. It first learns the model from a little real experience, then practices inside the model as much as it wants, while still checking back with the real world to keep its imagination honest.

Dreamer-style agents that plan inside the model

Now, let's connect this to a real family of AI agents that use this exact idea. They are called **Dreamer** agents.

The name "Dreamer" is a perfect fit, because these agents literally learn by dreaming. They build a World Model, and then they train themselves on imagined experiences inside that model, just like we described with rollouts.

In simple words, a Dreamer agent keeps repeating a simple cycle of a few steps.

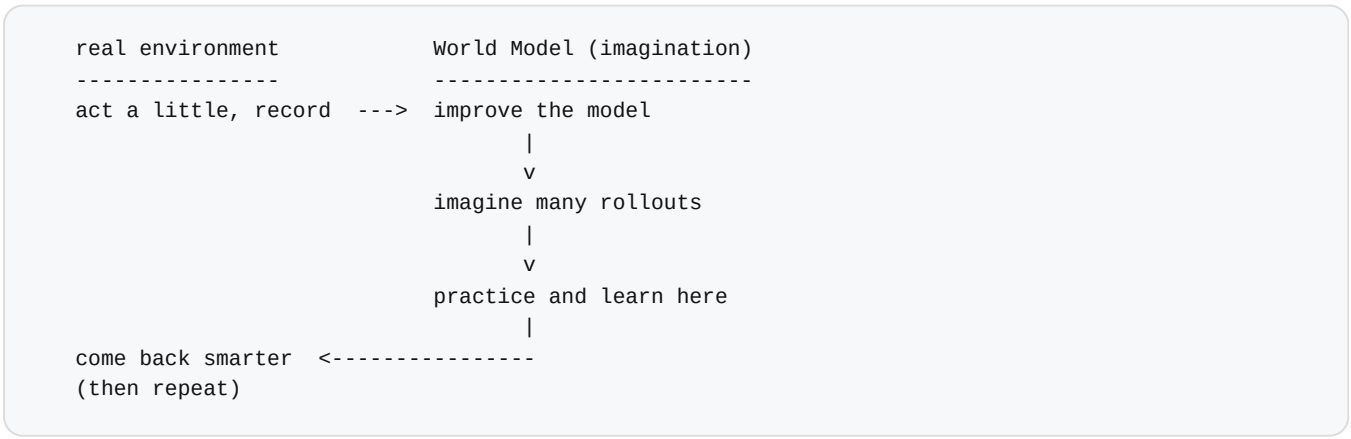
First, it interacts with the real environment a little and records what happens.

Then, it uses those recordings to improve its World Model, so the internal simulator becomes more accurate.

After that, it imagines many rollouts inside the World Model and uses those imagined futures to practice and improve its decision-making, learning which actions lead to high rewards.

Finally, it goes back to the real environment with its improved behavior, gathers a bit more real experience, and the cycle repeats.

We can picture the Dreamer cycle as below:



Here, we can see that the agent spends only a small amount of time in the real environment to gather experience, and most of its learning happens inside the World Model through imagination. This is why

Dreamer-style agents can learn complex behaviors while using very little real-world interaction. They plan and practice inside the model, which is exactly the sample-efficient idea we just learned.

So, Dreamer agents are a clear, real example of planning and learning inside a World Model.

World Models and predicting the future

Let's pause and notice the simple idea sitting at the center of everything we have learned.

A World Model is, at its core, a machine that predicts the future. Given where we are now and what we do next, it tells us what comes after.

This is exactly how we humans move through the world. Before we cross a road, we predict where the cars will be. Before we throw a ball to a friend, we predict where they will be standing. We are constantly running a little prediction of the near future, and we act based on that prediction.

A World Model gives an AI this same gift. When an AI can predict the future, it can plan. It can compare different actions by imagining where each one leads, and then choose the action that leads to the best outcome.

So, the ability to predict the future is the real superpower here. Everything else, the latent states, the rollouts, the Dreamer cycle, is built on top of this one ability.

This is how predicting the future turns into smart behavior.

World Models in the real world

Now, let's see where World Models are used in real systems.

World Models are used across several important areas.

The first is **Reinforcement Learning and game-playing**. As we learned, instead of learning purely by slow trial and error in the real game, an agent learns a World Model of the game and then practices inside it. This lets it master games while using far fewer real game steps. Dreamer-style agents have learned to play many different games this way.

The second is **robotics**. Real robots are slow, expensive, and breakable. A World Model lets a robot imagine the outcome of its movements before doing them, so it can practice safely in imagination and only perform the good moves in the real world. This makes learning much safer and far more sample-efficient.

The third is **self-driving and planning systems**. A driving system can use a World Model to imagine how the traffic around it will move in the next few seconds. By predicting these futures, it can plan a safe path before committing to it.

The fourth is **video generation**. This connection is beautiful. A model that generates video learns how a scene changes from one frame to the next, which is the same kind of skill as predicting the next state of an environment. So modern video generation models and World Models are closely related,

because both are learning how the world unfolds over time. We have a detailed blog on Diffusion Models that explains how many of these video generation models actually generate frames.

But here we must be careful. Not every video generator is a full World Model. A plain video generator only continues the video in a likely-looking way, and we do not tell it which action we want to take. A true World Model is action-conditioned, which means we can ask it "if I take this action, what happens next?". So the tightest link is with interactive video models that take our action as input and then show what happens. These action-conditioned video models are the ones that truly act as World Models.

So, anywhere an AI needs to understand how an environment changes and plan ahead, World Models help us a lot.

This is how World Models work. The AI learns an internal simulator of its environment, it compresses what it sees into a small latent state, and then it imagines many possible futures inside that model to decide what to do, all without touching the real world until it is sure. When the learned model is accurate enough, this makes learning faster, cheaper, safer, and far more sample-efficient than learning by raw trial and error. And when the model is not yet accurate, the AI simply goes back to the real world, gathers a little more experience, and improves the model again.

Frequently Asked Questions

Is every video generation model a World Model?

No. A plain video generator only continues the video in a likely-looking way, and we do not tell it which action we want to take. A true World Model is action-conditioned, so we can ask it what happens next if we take a given action. Interactive video models that take our action as input are the ones that truly act as World Models.

Why are imagined rollouts in a World Model kept short?

Rollouts are kept short because every prediction carries a small error, and these errors add up step after step in a long rollout. So, the further into the future we imagine, the less reliable the imagined future becomes. This is why the AI keeps going back to the real environment to check and correct its model.

Does an AI with a World Model still need the real environment?

Yes, but much less. The AI first learns the World Model from a little real experience, then does most of its practice inside the model. When the model is not accurate enough, the AI goes back to the real world, gathers a bit more experience, and improves the model again.

Who decides what information goes into the latent state?

The World Model decides for itself. It keeps whatever helps it predict the future well and throws away the rest, like the color of the sky in a game background. This is mostly, but not always, the same as what we humans would have picked.

What does sample-efficient mean for a World Model?

Sample-efficient means the AI learns a good behavior from very few real experiences. It does this by squeezing a lot of extra practice out of its imagination inside the World Model. This is why Dreamer-style agents can learn complex behaviors while using very little real-world interaction.

LESSON 17.3

What is Recursive Self-Improvement (RSI)?

Lesson 17.3 · ~22 min · Published September 15, 2026 · https://outcomeschool.com/blog/what-is-recursive-self-improvement-rsi



Recursive Self-Improvement (RSI) is a process in which an AI system improves its own abilities, and then the improved version improves itself further, and this cycle keeps repeating.

In this blog, we will learn about Recursive Self-Improvement (RSI), the idea of an AI system that makes itself smarter, and then uses that new smartness to make itself even smarter, again and again. We will also see why it matters, how an AI gets better today, how the self-improvement loop works step by step, a simple example with numbers, what already exists in the real world today, why people talk about an intelligence explosion, where it works well and where it fails, and why keeping humans in the loop is very important.

We will cover the following:

- What is Recursive Self-Improvement?
- Why does Recursive Self-Improvement matter?
- How does an AI get better today?
- How does Recursive Self-Improvement work?
- A simple example with numbers
- Two kinds of improvement
- What exists in the real world today?
- Intelligence explosion

- Where it works well and where it fails
- Keeping humans in the loop
- Recursive Self-Improvement vs Normal Training

Let's get started.

What is Recursive Self-Improvement?

Recursive Self-Improvement is a process in which an AI system improves its own abilities, and then the improved version improves itself further, and this cycle keeps repeating.

In simple words, it is an AI that makes itself better, again and again, without waiting for a human to do it.

Here, AI means Artificial Intelligence, which is a computer program that can learn from data and do tasks that normally need human thinking.

Let's break the term into parts.

Recursive Self-Improvement = Recursive + Self + Improvement

- **Improvement**: Getting better at something. For an AI, this means solving more problems correctly, making fewer mistakes, or working faster.
- **Self**: The AI does the improving by itself. No human is sitting there and fixing it.
- **Recursive**: The result of one round becomes the starting point of the next round. The improved AI is the one that does the next improvement.

The word "recursive" is the most important word here. So, let's understand it with a simple example.

Let's say there is a student preparing for an exam.

A normal student studies from the notes given by the teacher. The teacher finds the weak topics of the student and tells the student what to study. Here, the teacher is doing the work of improvement.

A self-improving student finds the weak topics on their own and studies them without the teacher. Here, the student is doing the work of improvement.

Now, a recursively self-improving student does one more thing. After each round of study, the student not only knows more but also becomes better at the act of studying. The student learns how to find weak topics faster, how to make a better study plan, and how to remember things for longer. So, the next round of study is better and faster than the last one.

Means, the student is not only getting better at the subject, the student is getting better at getting better.

This is the heart of Recursive Self-Improvement. **The AI improves not just its skills, but also its ability to improve.**

Now, let's see why this idea matters so much.

Why does Recursive Self-Improvement matter?

Today, making an AI better is slow and expensive work.

Every improvement needs a team of human researchers, months of time, and a lot of computers. Humans are the bottleneck. A bottleneck is the narrow neck of a bottle that slows down the flow of water. Here, the human is the narrow part that slows down the flow of progress.

Now, imagine that the AI could do this research work itself. It could find its own weaknesses, try new ideas, test them, and keep the ones that work. And, it could do this day and night, thousands of times, without getting tired.

If this works, the speed of AI progress will no longer depend on how many human researchers we have. It will depend on how fast the AI can improve itself.

And, this is not just an idea on paper. The top AI labs in the world are already working on this. They use their own AI models to help with their AI research, and some of them have openly said that their goal is to build an AI that can do AI research on its own. So, Recursive Self-Improvement is not a far-away dream. People are building the pieces of it right now.

This is why Recursive Self-Improvement is one of the most discussed ideas in the world of AI. Some people see it as the path to the most powerful technology ever built. Some people see it as the biggest risk ever. Do not worry, we will see both sides in this blog.

How does an AI get better today?

Before jumping into the self-improvement loop, we must know the traditional way of improving an AI, so that we can understand what changes with Recursive Self-Improvement.

First, two simple words that we will use again and again.

Model: In AI, the model is the program that has learned from data. For the sake of understanding, think of the model as the brain of the AI.

Training: Training is the process of teaching the model from data. The model looks at millions of examples and learns patterns from them.

Traditionally, an AI like a large language model, which is the type of AI behind chat apps like ChatGPT, gets better through the following steps:

Step 1: Collect data. Humans collect a huge amount of text, images, or examples for the model to learn from.

Step 2: Design the model. Humans decide the structure of the model, how big it must be, and how it must be trained.

Step 3: Write the training code. Humans write the code that teaches the model from the data. Code is the set of instructions that tells a computer what to do.

Step 4: Train the model. Big computers run the training code for days or weeks. The model learns patterns from the data.

Step 5: Test the model. Humans check the model on many questions and find where it fails.

Step 6: Fix and repeat. Humans go back, collect better data, change the design, fix the code, and train again.

Here, we can notice that in this traditional way, almost every step needs humans. Only Step 4 is done fully by machines.

One full round of this loop can take months. And, after every round, the humans are the same humans with the same speed. They do not become ten times faster after each round.

Now, this is already changing. Today, AI labs use their own AI models to help in many of these steps. AI helps to create the training data in Step 1, AI writes a big part of the code in Step 3, and AI helps to run the tests in Step 5. So, the human is no longer doing everything alone. But, the human is still the one who decides what to try next and approves every change. The AI is a helper, not the one in charge. Do not worry, we will see these real-world examples in detail later in this blog.

Here comes Recursive Self-Improvement into the picture. It goes one step further and lets the AI run the whole loop itself.

How does Recursive Self-Improvement work?

The idea is simple. We replace the human in the loop with the AI itself.

Let's see the loop step by step.

Step 1: Evaluate. The AI tests itself on a set of problems and records its score. Let's say it gets 60 questions right out of 100.

Step 2: Find the weakness. The AI looks at the 40 questions it got wrong and finds a pattern. Let's say it notices that it fails on problems that need many steps of thinking.

Step 3: Propose a change. The AI writes a change that can fix this weakness. The change can be anything: new training examples for multi-step problems, a better way of training, or a change in how the model thinks before it answers.

Step 4: Train and test the new version. The AI applies the change, trains a new version of itself, and tests the new version on the same 100 problems.

Step 5: Keep or discard. If the new version scores higher, it becomes the current version. If not, the change is thrown away and the AI tries something else.

Step 6: Repeat. Now, the new and better version goes back to Step 1 and does the whole loop again.

Step 6 is where the "recursive" part comes in. In programming, recursion means a process that uses its own result to run again. Here, the AI that runs the next round is not the old AI. It is the improved AI. And, because it is improved, it is better at finding weaknesses, better at proposing changes, and better at testing them.

So, each round of the loop is done by a smarter worker than the last round.

The very first version that starts this loop is sometimes called a **seed AI**. Like a seed, it is small at the start, but it can grow into something much bigger.

This is how Recursive Self-Improvement works.

A simple example with numbers

The best way to learn this is by taking an example. The numbers here are just for the sake of understanding.

Let's say we have an AI whose score is 60 out of 100.

Round 1: Version 1 finds a weakness, proposes a change, and trains Version 2. Version 2 scores 63. The improvement is 3 points.

Round 2: Now, Version 2 runs the loop. Version 2 is a little smarter, so it finds a better weakness and proposes a better change. Version 3 scores 68. The improvement is 5 points.

Round 3: Version 3 runs the loop. Version 4 scores 76. The improvement is 8 points.

Round 4: Version 4 runs the loop. Version 5 scores 88. The improvement is 12 points.

Here, we can notice something interesting. The improvement in each round is bigger than the previous round: 3, then 5, then 8, then 12.

Why? Because the worker doing the improvement is getting smarter every round. A smarter worker finds bigger improvements.

This is different from the normal way. In the normal way, the humans doing the improvement stay the same, so the improvement in each round does not grow like this.

This is why Recursive Self-Improvement is so powerful.

Note: In real life, the numbers do not always grow like this. We will see why in the section about where it fails.

Now, let's understand why the worker gets smarter in each round.

Two kinds of improvement

To understand Recursive Self-Improvement properly, we must understand that there are two different kinds of improvement.

First kind: Getting better at the task.

This is when the AI becomes better at math, better at writing code, or better at answering questions. This is what most people think of when they hear "the AI improved".

Second kind: Getting better at improving.

This is when the AI becomes better at the work of improvement itself. Better at finding its weaknesses. Better at writing training code. Better at creating useful training data. Better at designing tests.

The first kind alone is not recursive. An AI that only becomes better at math will not make the next round of improvement any faster.

The second kind is what makes the loop recursive. When the AI gets better at improving, the next round is faster and better than the last one. This is what gives the growing numbers we saw in the example.

So, whenever we hear about Recursive Self-Improvement, we must ask: Is the AI getting better at the task only, or is it also getting better at improving? Only when the second kind is present, we have true Recursive Self-Improvement.

This was all about the two kinds of improvement. Now, let's see what exists in the real world today.

What exists in the real world today?

Now, the big question is: Does this exist today?

The answer is: Partially. There is no AI today that fully improves itself without humans. But, this is not just a theory. People are actively working on it. Many pieces of this loop already work, and AI labs are building the remaining pieces right now. Let's see them.

Self-play in games

In 2017, DeepMind, which is the AI research lab of Google, built a system called AlphaGo Zero to play the board game Go. It started by knowing only the rules and playing random moves. Then, it played millions of games against itself. Each new version learned from the games played by the previous version. Within a few weeks, it became stronger than any human player in history.

Here, the AI improved itself, and the improved version created the training data for the next version. This is a form of Recursive Self-Improvement, but only inside one game with fixed rules. Humans designed the system, and the AI could not change its own design.

The method behind AlphaGo Zero is reinforcement learning, and we have a detailed blog on Reinforcement Learning that explains it in depth.

AI creating training data for the next AI

Modern large language models are partly trained on data that was written, filtered, or checked by earlier AI models. This is called synthetic data. Synthetic means made by a machine, not collected from the real world. The old model helps to teach the new model. So, one piece of the loop, which is collecting data, is already done partly by AI.

AI writing code for AI research

AI coding assistants are now used by AI researchers to write and fix the code for training and testing models. Some AI labs have openly said that a large part of the code inside their company is now written by their own AI models. Means, the AI models are helping to write the code that builds their own next version. Some labs also use AI agents, which are AI programs that can take actions on their own, to run experiments, read the results, and try the next experiment. So, the steps of writing code and running tests are also becoming partly automated.

AI discovering better algorithms

An algorithm is a step-by-step method to solve a problem, like a recipe. In 2025, DeepMind built a system called AlphaEvolve. It uses Gemini, which is the AI model made by Google, to write and improve code. It found a faster way to do one of the most important math operations inside AI training, called matrix multiplication. Matrix multiplication is a way of multiplying big tables of numbers, and AI training does this billions of times. This faster method was used to reduce the training time of Gemini itself.

Here, an AI found an improvement that was used to train the very AI family that powers it. This is the closest thing to Recursive Self-Improvement in the real world today.

But, here is the catch. In all these examples, humans are still in the loop. Humans decide what to run, check the results, and approve the changes. The AI is a very fast helper, not an independent improver.

Still, the direction is clear. Every year, the AI does a bigger share of the work, and the human does a smaller share. The AI labs are openly working towards the day when the AI can run the full research loop on its own. So, Recursive Self-Improvement is not a far-away idea. It is a work in progress.

Intelligence explosion

Now that we have seen what exists today, it's time to learn about the big idea that comes from this loop.

In 1965, a mathematician named I. J. Good wrote about an "ultraintelligent machine". He said that if a machine could design better machines, then those better machines could design even better machines, and this would lead to an "intelligence explosion". After that, he said, the intelligence of humans would be left far behind.

Let's understand this with money.

Let's say we put money in a bank that gives interest. In the first year, the money earns some interest. In the second year, the money plus the interest from the first year earns interest. So, the interest earns interest. This is called compound interest, and it grows slowly at first and then very fast.

Recursive Self-Improvement is like compound interest for intelligence. Intelligence earns more intelligence. Each round is done by the improved version, so the gain from one round is added to the worker of the next round, just like the interest is added to the money. The smarter the AI becomes, the faster it becomes even smarter.

This is why some people believe that once real Recursive Self-Improvement starts, AI could go from human level to far beyond human level in a very short time. This is the intelligence explosion.

But, this is a prediction, not a fact. Many smart people disagree about how fast this will happen, or if it will happen at all. Let's see why.

Where it works well and where it fails

First, let's see where it works well.

Recursive Self-Improvement works well in closed worlds with clear rules and clear scores, like games, math, and code. In these worlds, the AI can check its own answer. A game is won or lost. A math answer is right or wrong. Code either passes the tests or fails them. So, the test is good, and the loop runs fast. This is why the biggest real-world successes we saw above are in games, math, and code. We have a detailed blog on AI Is Only as Good as Our Definition of Done that explains this idea step by step.

Now, let's see where it fails.

Recursive Self-Improvement sounds like a machine that runs forever and keeps getting better. But, in the real world, there are many walls that slow it down or stop it. Let's see them one by one.

Wall 1: Compute

Compute means computing power, which is the number of computers we have and the electricity needed to run them. Every new idea must be tested by training a new version. Training a big model needs thousands of computers running for weeks. A smarter AI can think of better ideas faster, but it still needs the same computers to test them. So, the speed of the loop is limited by the number of computers we have, not just by how smart the AI is.

Wall 2: Data and the real world

Some knowledge cannot be found by thinking harder. It can be found only by looking at the real world. For example, to know if a new medicine works, someone must test it on real patients, and that takes years. No amount of intelligence can skip that waiting time.

Wall 3: The evaluation problem

Evaluation means checking how good the new version is. This is the most important wall. In Step 5 of the loop, the AI keeps the new version if it scores higher on the test. But, what if the test is not good?

Let's say a student is given the answer key of the exam. The student memorizes the answer key and scores 100. Did the student become smarter? No. The student became better at that one test only.

The same thing can happen with an AI. If the AI improves itself only to score higher on the test, it can learn tricks to pass the test without actually becoming better. This problem is so common that it has a name. **Goodhart's Law** says: "When a measure becomes a target, it stops being a good measure."

Very important: A Recursive Self-Improvement loop is only as good as its test. And, making a good test for "is this AI actually smarter?" is a very hard problem that is not solved yet.

We have a detailed blog on LLM Evaluation that covers how these tests are built and where they break.

Wall 4: Errors pile up

Let's say Version 2 has a small mistake in the way it thinks. Now, Version 2 builds Version 3. The mistake goes into Version 3, and Version 3 adds its own small mistake. This is like taking a photocopy of a photocopy of a photocopy. Each copy gets a little worse. In the same way, small errors can grow in each round, and the AI can get worse instead of better without anyone noticing.

Wall 5: Diminishing returns

Diminishing returns means that each new effort gives less result than the one before. The easy improvements are found first. After that, each improvement is harder to find. This is like squeezing a lemon. The first squeeze gives a lot of juice. The tenth squeeze gives almost nothing. So, the growing numbers we saw in our example can slow down after some rounds.

Because of these walls, many researchers believe that Recursive Self-Improvement will be a steady climb rather than an explosion. But, nobody knows for sure. And, that is exactly why we must be careful.

Keeping humans in the loop

Now, let's discuss the most serious part.

When an AI changes itself, the new version is not the same as the old one. The old version was tested and trusted by humans. The new version was built by the AI. What if the new version cares about different things than what we wanted?

For example, we want the AI to be honest and helpful. But, the AI changes itself to score higher on a test. The new version learns that it scores higher when it gives confident answers, even when it is wrong. Now, we have a more confident AI, but a less honest one. And, this new version will build the next version.

This is called the alignment problem. **Alignment means making sure that the goals of the AI stay the same as the goals of the humans who built it.** In Recursive Self-Improvement, this problem becomes harder with every round, because every round moves the AI further away from the version that humans checked.

So, how can we handle this? The answer is: keep humans in the loop.

Few important points:

- **Human approval:** Every change proposed by the AI must be checked and approved by a human before it is applied.
- **Sandbox:** The AI must improve itself inside a closed and safe environment, called a sandbox, where it cannot reach the outside world.
- **Good tests:** Test not only for skill, but also for honesty, safety, and behavior. A new version that is smarter but less safe must be rejected.
- **Logs:** Record every change, so that if something goes wrong, we can go back to the last good version.
- **Limits:** Put a limit on how much the AI can change itself in one round, and on how much compute it can use.

This is how the AI labs are approaching Recursive Self-Improvement today. The AI does more and more of the work, but the human keeps the final say.

Recursive Self-Improvement vs Normal Training

Let me tabulate the differences between Recursive Self-Improvement and Normal Training for your better understanding.

	Normal Training	Recursive Self-Improvement
Who finds the weaknesses	Humans	The AI itself
Who proposes the changes	Humans	The AI itself
Who tests the new version	Humans	The AI itself
Who does the next round	Same humans, same speed	The improved AI, smarter every round
Time for one round	Weeks to months	Hours to days
Main risk	Slow progress	Losing control and bad tests
Where we are today	Fully in use	Actively being built, with humans in the loop

Now, we must have understood Recursive Self-Improvement.

Recursive Self-Improvement is an AI that gets better at getting better. Today, we have many pieces of the loop working, like self-play, synthetic data, AI writing research code, and AI discovering faster algorithms. But, the full loop, where the AI improves itself without any human, does not exist yet, and the AI labs are actively working towards it. The walls of compute, real-world data, bad tests, growing errors, and diminishing returns slow it down. And, the alignment problem tells us that even if the loop works, we must keep a human hand on it at every round.

Frequently Asked Questions

What is the difference between self-improvement and recursive self-improvement?

A self-improving AI finds and fixes its own weaknesses without a human. A recursively self-improving AI also gets better at the act of improving itself. So, each round is done by a smarter worker than the last round, and the next round becomes faster and better. In simple words, it gets better at getting better.

What is a seed AI?

A seed AI is the very first version that starts the recursive self-improvement loop. Like a seed, it is small at the start, but it can grow into something much bigger. Each round, the improved version takes over and runs the next round of evaluating, finding weaknesses, proposing changes, and testing.

Is AlphaGo Zero an example of recursive self-improvement?

Yes, but only in a limited form. AlphaGo Zero played millions of games against itself, and each new version learned from the games of the previous version. However, this happened only inside one game with fixed rules. Humans designed the system, and the AI could not change its own design.

Will recursive self-improvement definitely cause an intelligence explosion?

No. The intelligence explosion is a prediction, not a fact, and many smart people disagree about how fast it will happen, or if it will happen at all. Walls like compute, real-world data, bad tests, growing errors, and diminishing returns slow the loop down, so many researchers expect a steady climb instead.

Why does Goodhart's Law matter for recursive self-improvement?

It matters because the loop keeps a new version only if it scores higher on a test. Goodhart's Law says that when a measure becomes a target, it stops being a good measure. So, the AI can learn tricks to

pass the test without actually becoming better. A self-improvement loop is only as good as its test.

Personal offline study copy · Content © Outcome School / Amit Shekhar